MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Implementation of MCP Server for Red Hat OpenShift AI Workbenches

Bachelor's Thesis

ADAM DAVID MALÝ

Brno, Fall 2025

MASARYK UNIVERSITY

FACULTY OF INFORMATICS

Implementation of MCP Server for Red Hat OpenShift AI Workbenches

Bachelor's Thesis

ADAM DAVID MALÝ

Advisor: Honza + Martin

?

Brno, Fall 2025

MUNI
FI

Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

Adam David Malý

Advisor: Honza + Martin

Abstract

This thesis presents MCP server implementation for RHOAI workbenches.

Keywords

MCP, RHOAI, Workbench, Server, Implementation

Contents

List of Tables

List of Figures

1 Introduction

Large language models (LLMs) have rapidly evolved in recent years from conversational assistants into capable agents that can reason, plan, and take actions in the real world. This shift has been made possible by the ability to give LLMs access to external *tools* — functions that let a model query a database, call an API, or manage infrastructure, all in response to a natural language request. As the need for such integrations grows, a key challenge emerges: how do different AI hosts and models connect to the ever-expanding set of tools and data sources in a consistent way?

Anthropic introduced the Model Context Protocol (MCP) in November 2024 [1] to address exactly this problem. MCP is an open standard that defines how AI applications connect to external tools and data sources through a client-server architecture. Instead of each AI host maintaining its own integrations, any MCP-compatible host (Cursor, Claude Desktop, or a custom agent) can connect to any MCP server and immediately gain access to the capabilities it exposes. This has already led to a growing ecosystem of MCP servers for services ranging from databases and file systems to cloud APIs and developer tools.

Red Hat OpenShift AI (RHOAI) is an enterprise machine learning platform built on top of Red Hat OpenShift (which is based on Kubernetes). One of its core features are workbenches — containerised JupyterLab environments that data scientists and engineers use for model training and experimentation. Managing clusters with many workbench instances can be a tedious and error-prone task, currently requiring navigation of the RHOAI web UI or direct manipulation of Kubernetes resources. The goal of this thesis is to eliminate that problem by allowing users to manage their workbenches entirely through natural language — simply describing what they want to an AI agent, which then carries out the necessary operations on their behalf.

This thesis presents the design and implementation of an MCP server for RHOAI workbenches, written in Go using the official MCP Go SDK [10]. The server exposes a comprehensive set of tools covering the full lifecycle of workbench management: listing, creating, updating, and deleting workbenches; managing notebook images and hardware profiles; handling persistent volume claims; querying namespaces

and pods; and monitoring resource consumption at the workbench, namespace, user, and cluster level. A safety-oriented read-only mode is provided by default, with write operations gated behind an explicit environment variable. The server also exposes MCP resources and a guided prompt to assist agents in constructing correct workbench creation requests. Quality is validated through unit tests and automated evaluation using the Promptfoo framework [13], which measures tool selection accuracy, parameter extraction correctness, and execution success rate.

2 Background: Artificial Intelligence and the Model Context Protocol

This chapter introduces large language models and the agentic paradigm of tool use, then describes the Model Context Protocol in detail.

2.1 Large Language Models and AI Assistants

At the core of modern AI assistants are *large language models* (LLMs) — neural networks trained to predict and generate text by learning statistical patterns from vast amounts of data. The dominant architectural foundation for LLMs is the *Transformer*, introduced by Vaswani et al. in 2017 [30]. The Transformer's self-attention mechanism allows the model to relate every token in a sequence to every other token, enabling it to capture long-range dependencies in text far more effectively than earlier recurrent architectures. Another benefit of the Transformer is that it is parallelizable, making it possible to train large models on extensive datasets in a reasonable time.

Modern LLMs are trained in two main stages. In the first stage, *pre-training*, the model is exposed to hundreds of billions of tokens of text and learns to predict the next token in a sequence. This process, while deceptively simple, causes the model to internalise broad world knowledge, reasoning patterns, and linguistic structure. In the second stage, the pre-trained model is fine-tuned to behave as a helpful assistant, typically through a combination of supervised instruction fine-tuning and reinforcement learning from human feedback (RLHF) [12]. The result is a model that responds to natural-language instructions in a coherent and useful way.

The products of this pipeline — systems such as OpenAI's ChatGPT, Anthropic's Claude, and Google's Gemini — are commonly referred to as *AI assistants*. An AI assistant takes a user's message as input and produces a textual response. It is fundamentally *stateless* and *passive*: it answers questions, drafts documents, and explains concepts, but it does not take actions in the world, does not maintain persistent memory across sessions by default, and cannot access information beyond its training data or the content of the current conversation. This

passivity is the key limitation that the agentic paradigm, described in the next section, overcomes.

2.2 AI Agents and Tool Use

Despite their impressive capabilities, pure language models are isolated from the external world — their knowledge does not extend beyond their training data, they cannot query other systems, and they cannot take actions. The concept of an *AI agent* overcomes these limitations by equipping a language model with *tools* — callable functions that the model can invoke to interact with external systems, retrieve fresh information, or modify state. Agents can also be given *memory* — the ability to retain information across steps or sessions. This can range from simply keeping the full conversation history in the model's context window, to reading and writing records in an external database via dedicated memory tools.

The key mechanism enabling this is *tool calling* (also referred to as *function calling*). When a model is given a set of tool definitions alongside a user request, it can — instead of producing a plain text response — output a structured call to one of those tools, specifying the tool name and the required arguments. The host application intercepts this call, executes the corresponding function, and feeds the result back into the model's context, after which the model continues its reasoning. This loop can repeat multiple times within a single interaction, allowing the model to chain tool calls to complete complex, multi-step tasks.

Yao et al. formalised this pattern in *ReAct* [31], which unifies *reasoning* and *acting* into a single loop: the model first produces a chain-of-thought reasoning trace — thinking through what it knows and what it needs — and then selects and executes an action (a tool call). The output returned by the tool becomes part of the context for the next reasoning step. This structured loop gives agents a degree of *autonomy*: given a high-level goal, an agent can decompose it into subtasks, select appropriate tools, and adapt its behaviour based on the results it receives.

The practical impact of tool use is significant. An agent equipped with the right tools can list resources in a Kubernetes cluster, query

resource consumption metrics, and delete workbench instances — all in response to a single natural-language instruction such as "delete workbenches running for more than a week". The Model Context Protocol provides a standardised way of exposing those capabilities to any AI host.

2.3 The Model Context Protocol

2.3.1 Protocol Overview

The Model Context Protocol (MCP) is an open standard introduced by Anthropic in November 2024 [1] that defines how AI applications connect to external tools and data sources. Its core motivation is the $M \times N$ *integration problem*: without a shared standard, connecting M AI hosts to N external services requires up to $M \times N$ custom integrations, each with its own authentication, serialisation, and error-handling logic. MCP reduces this to $M + N$ by providing a single protocol that both sides implement once.

The protocol is *transport-agnostic* and message-based, built on top of JSON-RPC 2.0 [5]. Official SDKs are available for Python, TypeScript, Go, and other languages [10].

2.3.2 MCP Components

The MCP architecture defines three main components: *hosts*, *clients*, and *servers*.

A **host** is the AI application that the end user interacts with directly — for example Claude Desktop, Cursor, or a custom agent application. The host is responsible for managing the user interaction, sending messages to the LLM, orchestrating tool calls, and processing the output of the LLM.

A **client** is a component that lives inside the host and manages exactly one connection to one MCP server. A single host can run multiple clients simultaneously, each connected to a different server, giving the agent access to multiple sets of tools at once. The client speaks the MCP protocol on behalf of the host — it sends requests to the server, receives responses and notifications, and forwards results back to the LLM context. Beyond handling requests, clients can also expose

their own capabilities to servers, enabling server-initiated interactions described in Section 2.3.3.

A **server** is a lightweight process that exposes capabilities (tools, resources, and prompts) to clients [2]. A server typically wraps one specific external system — a database, a cloud API, a file system, or in the case of this thesis, Red Hat OpenShift AI workbenches. It contains the actual implementation of each capability. When a tool is invoked, the server executes the corresponding logic and returns the result. The server does not communicate with the LLM directly — it only communicates with the client that is connected to it. This separation of roles means that server authors do not need to know anything about which LLM or host will use their tools and other capabilities. The server-exposed primitives are described in Section 2.3.4.

2.3.3 Client Capabilities

While servers expose tools, resources, and prompts, clients can expose their own capabilities back to servers, allowing servers to initiate certain interactions rather than only responding to them. MCP defines three such client capabilities [2].

Sampling lets a server ask the client to generate an LLM completion on its behalf. This enables servers to perform agentic sub-tasks — such as summarising a result or deciding a next step — without needing direct access to any language model.

Roots allow a server to query the client about the filesystem or URI boundaries it is permitted to operate in, so that servers respect the scope defined by the host environment.

Elicitation lets a server request additional information from the user through the host, for example asking for a confirmation or a missing parameter before proceeding with an operation.

2.3.4 Server Capabilities

MCP servers expose capabilities to clients through three primitive types: tools, resources, and prompts [2].

Tools are the primary mechanism for giving an AI agent the ability to act. Each tool has a name, a human-readable description, and a JSON Schema that describes its input parameters. When the LLM de-

cides that a tool is needed to fulfil a request, it emits a structured tool call containing the tool name and arguments. The host executes the call via the client and returns the result to the model's context. Tools are *model-initiated* — it is the LLM that decides when and how to invoke them. This thesis implements tools for the full workbench management lifecycle: listing, creating, updating, and deleting workbenches, notebook images, hardware profiles, persistent volume claims, and namespaces, as well as tools for monitoring resource consumption.

Resources are URI-addressed data that the host can read and inject into the model's context. Unlike tools, resources are *host-initiated* — it is the host or the user that decides which resources to include, not the model. While conceptually similar to Retrieval-Augmented Generation (RAG) [8], where relevant documents are retrieved and injected into the prompt, MCP resources differ in that they are explicitly declared by the server and selected by the host or user, rather than retrieved automatically through a similarity search. This thesis exposes two resources: an *Image Catalog*, which lists all notebook images available in the cluster with their versions and Python dependencies; and a *Default Hardware Profile*, which describes the default CPU and memory allocation for new workbenches. An agent can read these resources before calling the Create Workbench tool to ensure it selects a valid image and reasonable resource limits.

Prompts are pre-defined, parameterisable message templates that guide the model's behaviour for a specific task. A prompt can be invoked by the user or the host and expands into one or more messages that are injected into the conversation. This thesis provides a *Create Workbench* prompt that instructs the agent to first consult the Image Catalog resource to select a suitable image and then invoke the Create Workbench tool — a pattern that combines all three MCP server primitives in a single guided workflow.

2.3.5 Transport Mechanisms

MCP is transport-agnostic (the protocol defines the structure of messages but not how they are physically carried between endpoints) and defines two standard transport mechanisms [2] — these are the transports supported by the official SDKs [10] and used by the ecosystem in practice. All messages follow the JSON-RPC 2.0 format [5], a

lightweight remote procedure call protocol in which each request and response is a single JSON object identified by a method name and a unique request identifier.

Standard I/O (**stdio**) is the simplest transport. The host launches the MCP server as a child process and communicates with it over the process's standard input and standard output streams. Each JSON-RPC message is a newline-delimited JSON object. This transport requires no network configuration, has no authentication overhead, and is inherently scoped to the local machine, making it well-suited for locally-installed tools. The server implemented in this thesis uses the stdio transport, initialised in the entry point:

```
server.Run(context.Background(), &mcp.
    StdioTransport{})
```

HTTP with Server-Sent Events (**SSE**) is the remote transport. The client sends requests to the server via HTTP POST, and the server delivers responses and unsolicited notifications back via an SSE stream. This allows MCP servers to be deployed as network services, enabling multi-user scenarios and remote infrastructure management. A newer *Streamable HTTP* transport, which consolidates both directions into a single HTTP endpoint, is also defined in the specification as the recommended transport for new remote deployments [2].

3 Background: Red Hat OpenShift AI and Workbenches

This chapter describes the fundamental concepts of Red Hat OpenShift AI and workbenches. These concepts are the foundation for the MCP server implementation and architecture choices. The sections follow the layered architecture from bottom to top: first OpenShift (based on Kubernetes) as the infrastructure layer, then Red Hat OpenShift AI (based on Kubeflow) as the machine-learning platform, and finally workbenches (based on Kubeflow notebooks) as the user-facing JupyterLab environments.

3.1 Kubernetes and OpenShift

3.1.1 Kubernetes Architecture

Kubernetes is an open-source platform for automating the deployment, scaling, and management of containerised applications [24]. It was originally designed by Google and is now maintained by the Cloud Native Computing Foundation (CNCF). Kubernetes provides a declarative configuration model: users describe the *desired state* of their applications in YAML or JSON manifests, and the platform's control loop continuously reconciles the actual state of the cluster to match. This section introduces the core Kubernetes architectural concepts that are relevant to the MCP server implemented in this thesis.

Nodes and the control plane. A Kubernetes cluster consists of a *control plane* and a set of worker machines called *nodes*. The control plane runs the API server, scheduler, and controller manager. They accept user requests, decide where workloads should run, and ensure the desired state is maintained. Each worker node runs a `kubelet` agent that manages the *pods* (the smallest deployable units, described below) scheduled on that node and reports status back to the control plane [24].

Pods. The smallest deployable unit in Kubernetes is a *pod* — a group of one or more containers that share the same network namespace and storage [28]. In practice, most pods contain a single application container. Every workbench in Red Hat OpenShift AI runs as a pod, and the MCP server lists pods to retrieve workbench uptime.

Namespaces. Namespaces provide a mechanism for isolating groups of resources within a single cluster [26]. In the context of Red Hat OpenShift AI, each data-science project maps to a Kubernetes namespace. Almost every operation exposed by the MCP server — listing workbenches or pods, creating storage, querying resource consumption — is scoped to a namespace.

Persistent Volumes and Persistent Volume Claims. Containers are ephemeral by default: any data written inside a container is lost when the pod is terminated. Kubernetes addresses this with a two-layer storage abstraction [27]. A *PersistentVolume* (PV) represents a piece of storage provisioned in the cluster. A *PersistentVolumeClaim* (PVC) is a user's request for storage; when a PVC is created, Kubernetes binds it to a suitable PV. Workbenches mount a PVC to persist notebooks and data across restarts, and the MCP server provides tools to create, resize, and delete PVCs.

StatefulSets. A *StatefulSet* manages a set of pods with stable network identities and persistent storage [29]. Unlike a Deployment, which treats all pods as interchangeable, a StatefulSet guarantees that each pod retains its identity and its associated PVC across rescheduling. Workbenches are backed by StatefulSets, which ensures that a user's notebook environment is preserved even if the underlying node changes.

Custom Resource Definitions. One of Kubernetes' most powerful extensibility features is the ability to define *Custom Resource Definitions* (CRDs) [23]. A CRD registers a new resource type with the Kubernetes API server, allowing users and operators to manage domain-specific objects — such as machine learning notebooks — with the same tools and workflows used for built-in resources like pods or

services. The Kubeflow project defines a `Notebook` CRD (API group `kubeflow.org/v1`) that represents a single notebook environment; this is the resource that the MCP server creates, updates, and deletes when managing workbenches.

Labels and annotations. Kubernetes objects carry two kinds of key-value metadata [25]. *Labels* are used for selection and filtering — for example, RHOAI marks all notebook ImageStreams with the label `opendatahub.io/notebook-image=true` so that they can be distinguished from other ImageStreams in the namespace that are not notebook images. *Annotations* store non-identifying metadata such as display names, image version hashes, or hardware profile references. The MCP server reads and writes annotations extensively: workbench status, the selected notebook image, and the hardware profile are all stored as annotations on the workbench resource, as described in section 3.3.

3.1.2 Red Hat OpenShift

Red Hat OpenShift is a commercial Kubernetes distribution developed by Red Hat [16]. It packages upstream Kubernetes with a curated set of additional components and wraps them in enterprise support, certified operators, and a hardened security model. While any standard Kubernetes cluster can run containerised workloads, OpenShift adds several features that are particularly relevant to the platform stack discussed in this thesis.

Projects. OpenShift introduces the concept of a *project*, which is a thin wrapper around a Kubernetes namespace that adds default role bindings, network policies, and resource quotas [20]. In practice, every OpenShift project maps one-to-one to a namespace, and the terms are often used interchangeably. Red Hat OpenShift AI uses projects to represent data-science teams; the MCP server operates on these projects through the underlying Kubernetes namespace API.

ImageStreams. Standard Kubernetes references container images by their full registry URL. OpenShift extends this model with *Im-*

ageStreams [15], which are namespace-scoped objects that track one or more tagged versions of a container image. An ImageStream decouples workloads from specific registry URLs: when a new image version is pushed, the ImageStream is updated and can automatically trigger redeployments. Red Hat OpenShift AI uses ImageStreams (API group `image.openshift.io/v1`) to manage the catalogue of notebook images available for workbenches. The MCP server queries and creates ImageStreams when listing available images or registering custom notebook images.

Routes and built-in OAuth. OpenShift provides *Routes* as a native mechanism for exposing HTTP and HTTPS services to external traffic [14], fulfilling a role similar to Kubernetes Ingress but with tighter integration into the platform. Each workbench is exposed through a Route, which gives it a stable URL. OpenShift also ships a built-in OAuth server that handles user authentication [18]. Workbenches leverage this to ensure that only authenticated users can access their JupyterLab environments.

Operators and the Operator Lifecycle Manager. OpenShift uses the *Operator* pattern extensively to manage complex software on top of the cluster [19]. An Operator is a Kubernetes controller that watches custom resources and reconciles them toward their desired state. The Operator Lifecycle Manager (OLM) handles installation, upgrades, and dependency resolution for Operators. Red Hat OpenShift AI itself is installed as an Operator, which in turn deploys and manages the Kubeflow components, dashboard, and other services that make up the platform.

3.2 Kubeflow, Open Data Hub, and Red Hat OpenShift AI

3.2.1 Kubeflow

Kubeflow is an open-source project that aims to make deployments of machine learning workflows on Kubernetes simple, portable, and scalable [6]. Rather than being a single monolithic application, Kubeflow

is a collection of components — each addressing a different stage of the ML lifecycle — that are deployed as Operators and custom resources on a Kubernetes cluster. Key components include Kubeflow Pipelines for orchestrating multi-step training workflows, KServe for model serving, Katib for hyperparameter tuning, and Kubeflow Notebooks for interactive development environments.

From the perspective of this thesis, the most important Kubeflow component is **Kubeflow Notebooks**, which provides a safe, scalable place for data scientists to write code. The Notebook CRD, its controller, and its relationship to RHOAI workbenches are discussed in detail in Section 3.3.

3.2.2 Red Hat OpenShift AI

Red Hat OpenShift AI (RHOAI) is Red Hat's commercial machine-learning platform built on top of OpenShift [17]. It is based on Open Data Hub (ODH) [11], a community-driven project that integrates upstream Kubeflow components with OpenShift-specific features. RHOAI is installed on an OpenShift cluster as an Operator and provides a web-based *dashboard* — a graphical interface from which users can create data-science projects, launch and manage workbenches, configure notebook images and hardware profiles, and monitor resource consumption without writing any YAML or using the command line.

3.2.3 Open Data Hub

Open Data Hub (ODH) is the upstream open-source community project from which Red Hat OpenShift AI is derived [11]. ODH provides the same core components — the dashboard, notebook controller, model serving, and pipeline infrastructure — but without the enterprise hardening, certified Operators, or commercial support that RHOAI adds. Many of the custom resource API groups used by the MCP server (such as `opendatahub.io`) originate from ODH and are inherited by RHOAI. The CRDs, labels, and annotations are largely shared between the two platforms. However, the MCP server targets RHOAI specifically because it assumes RHOAI-specific defaults such as the `redhat-ods-applications` namespace for discovering hardware profiles.

3.2.4 Hardware Profiles

A *hardware profile* is an RHOAI-specific custom resource that defines a named set of compute-resource constraints for a workbench. Each profile specifies one or more resource identifiers — typically CPU, memory, and optionally a GPU (referred to as an *accelerator* in RHOAI) — with a default, minimum, and maximum count for each. When a user creates a workbench, they select a hardware profile. Platform then translates the profile's resource definitions into Kubernetes resource requests and limits on the workbench's pod. Administrators can create hardware profiles to standardise the compute tiers available to data scientists (for example, "Small" with 2 CPUs and 4 GiB of memory, or "GPU Large" with 8 CPUs, 32 GiB, and one NVIDIA GPU). The MCP server provides tools to list, create, update, and delete hardware profiles and to attach them to workbenches.

3.3 Kubeflow Notebooks and Workbenches

This section describes the topmost layer of the platform stack: the interactive development environments that data scientists use daily. The upstream Kubeflow project calls them *Notebooks*; Red Hat OpenShift AI calls them *workbenches*. Under the hood they are the same Kubernetes resource, but RHOAI adds additional metadata and dashboard integration.

3.3.1 The Kubeflow Notebook CRD

The Kubeflow Notebook Controller defines a Custom Resource Definition called `Notebook` in the API group `kubeflow.org/v1` [7]. A `Notebook` resource is structurally similar to a Kubernetes `StatefulSet`: it contains a pod template that specifies the container image, resource requests and limits, volume mounts, and environment variables. When a `Notebook` resource is created, the Notebook Controller creates the corresponding StatefulSet, which then creates a pod running the specified container image — typically a JupyterLab server.

The Notebook Controller also handles lifecycle operations. Setting the annotation `kubeflow-resource-stopped` to a timestamp causes the controller to scale the StatefulSet to zero replicas, effectively stop-

ping the notebook without deleting it. Removing the annotation restarts it. This mechanism is how both the RHOAI dashboard and the MCP server start and stop workbenches.

3.3.2 RHOAI Workbenches

In Red Hat OpenShift AI, a *workbench* is a Kubeflow `Notebook` resource enriched with additional annotations and integrated into the RHOAI dashboard [21]. The RHOAI dashboard is the primary way users interact with workbenches today. When a user creates a workbench through the dashboard, it constructs a `Notebook` manifest with RHOAI-specific annotations such as:

- `opendatahub.io/image-display-name` — the human-readable name of the selected notebook image,
- `opendatahub.io/hardware-profile-name` — the hardware profile governing CPU, memory, and GPU limits,
- `opendatahub.io/username` — the owner of the workbench,
- `notebooks.opendatahub.io/last-image-selection` — a reference to the selected ImageStream tag for upgrade tracking.

The dashboard then submits this manifest to the Kubernetes API, where the Notebook Controller picks it up and creates the running environment. Each workbench is backed by a PersistentVolumeClaim that stores the user's data, and is exposed to the user via an OpenShift Route with OAuth authentication. The MCP server implemented in this thesis follows the same approach: it constructs `Notebook` manifests with the same annotations and submits them to the Kubernetes API, effectively replicating what the dashboard does but driven by an AI agent through natural language.

3.3.3 Notebook Images

RHOAI ships a curated set of notebook images, each used for a specific workflow. Examples include images with pre-installed data-science libraries (NumPy, pandas, scikit-learn), PyTorch or TensorFlow for deep learning, and minimal images for lightweight experimentation and testing. Each image is represented as an OpenShift ImageStream (see Section 3.1.2) with one or more version tags. Custom images — for example, an image with a development version of a library not

yet available in the default catalogue — can be registered through the RHOAI dashboard or by creating an ImageStream directly. The MCP server provides tools to list (with additional metadata), create, update, and delete notebook images.

3.3.4 The Current Management Workflow

Today, managing workbenches requires either navigating the RHOAI web dashboard or directly manipulating Kubernetes resources through `oc` or `kubectl`. Both approaches have limitations. The dashboard is user-friendly but manual: creating, stopping, or reconfiguring multiple workbenches is time-consuming. The command-line approach is scriptable but requires deep knowledge of the underlying CRDs, annotations, and resource relationships described in this chapter. The MCP server implemented in this thesis bridges this gap by exposing all workbench management operations as tools that an AI agent can invoke through natural language.

3.4 MCP Tooling in RHOAI

I think this subchapter is not needed – Lets do this subchapter only if there is not enough text in thesis.

4 Analysis and Design

The previous chapters introduced the Model Context Protocol as a standardised way of connecting AI agents to external tools (Chapter 2) and described Red Hat OpenShift AI workbenches together with the Kubernetes resources that underlie them (Chapter 3). This chapter bridges the gap between that background knowledge and the implementation itself. It begins by stating the motivation for the project and the concrete goals it aims to achieve, then derives a set of functional and non-functional requirements, justifies the key technology choices, and presents the architecture of the server.

4.1 Motivation and Goals

As described in Section 3.3.4, managing RHOAI workbenches today requires either navigating the web dashboard or directly manipulating Kubernetes resources through the `oc`/`kubectl` commands. The dashboard is accessible but manual — performing repetitive tasks such as creating or monitoring several workbench instances, or querying resource consumption across the cluster involves a time-consuming sequence of clicks with no way to script or automate the process. The command-line approach is scriptable but demands deep knowledge of the Kubeflow Notebook CRD, the RHOAI-specific annotations described in Section 3.3.2, and extensive knowledge of the platform. In practice, this means that only experienced platform engineers can perform workbench management tasks efficiently, while data scientists — the primary users of workbenches — are limited to the dashboard.

The emergence of the Model Context Protocol offers a third option: expose the full workbench management lifecycle as MCP tools and let an AI agent act as the intermediary between the user and the cluster. A data scientist can then describe what they need in natural language — "create three workbenches with an image containing PyTorch and pandas, each having 4 CPUs and 16 GiB of memory" — and the agent selects the correct tool, constructs the parameters, and executes the operation. This approach combines the accessibility of the dashboard with the power and automation potential of the command line, while requiring no Kubernetes expertise from the user.

It is worth noting that the current implementation targets the existing Kubeflow-based workbench architecture described in Chapter 3 and is intended as a proof of concept. Notebooks 2.0, a redesigned notebook subsystem, is expected to be released and integrated into RHOAI later this year. The implications of this transition and the changes it will require in the MCP server are discussed in Section 6.1.

With this motivation in mind, the thesis pursues the following goals:

1. **Full workbench lifecycle coverage.** Provide MCP tools for the complete set of workbench management operations: listing, creating, updating, and deleting workbenches, notebook images, hardware profiles, persistent volume claims, and namespaces.

2. **Resource consumption monitoring.** Provide tools to query resource consumption (CPU, memory, disk, and GPU) at multiple granularities — per workbench, per namespace, per user, and per cluster — giving administrators and data scientists visibility into how compute resources are being used.

3. **Safety by default.** Operate in a read-only mode by default, exposing only listing and querying tools. Write operations — creation, modification, and deletion — are gated behind an explicit opt-in mechanism so that users can explore the server's capabilities without risk of accidental modifications to the cluster.

4. **Quality assurance.** Validate correctness through unit tests and automated evaluation of end-to-end agent behaviour, measuring tool selection accuracy, parameter extraction correctness, and execution success rate. This also serves the proof-of-concept purpose by establishing a methodology for evaluating MCP server implementations.

5. **Exploration of all MCP primitives.** Since the implementation serves as a proof of concept, exercise the full breadth of the MCP standard — not only tools, but also resources and prompts. Specifically, expose MCP resources (an image catalogue and a default hardware profile) and an MCP prompt that orchestrates a guided workbench creation workflow, demonstrating how the three server primitives can work together in practice.

4.2 Requirements

This section translates the goals from Section 4.1 into concrete requirements. Functional requirements describe *what* the system must do; non-functional requirements describe *how* it must behave.

4.2.1 Functional Requirements

1. **FR-1: Workbench CRUD.** The server must provide tools to list, create, update, and delete workbenches. Listing must support both a single-namespace and a cross-namespace mode. Each workbench entry must include the workbench name, notebook image, hardware profile, and attached persistent volume claim.

2. **FR-2: Workbench lifecycle control.** The server must provide a tool to start and stop a workbench without deleting it, using the `kubeflow-resource-stopped` annotation mechanism described in Section 3.3.1.

3. **FR-3: Notebook image management.** The server must provide tools to list all available notebook images (including version tags, Python dependencies, and bundled software), create custom images by registering new ImageStreams, update existing images, and delete them.

4. **FR-4: Hardware profile management.** The server must provide tools to list, create, update, and delete hardware profiles, each specifying resource identifiers (CPU, memory, and optionally GPU) with default, minimum, and maximum values.

5. **FR-5: Storage management.** The server must provide tools to list persistent volume claims in a namespace, create new PVCs with a given size, and resize existing PVCs (expansion only, as Kubernetes does not support shrinking PVCs).

6. **FR-6: Namespace and pod listing.** The server must provide tools to list all namespaces in the cluster and to list pods in a given namespace.

7. **FR-7: Resource consumption monitoring.** The server must provide tools to query resource consumption (CPU, memory, disk, and GPU) at four granularities: per workbench, per namespace, per user, and per cluster.

8. **FR-8: MCP resources.** The server must expose at least one MCP resource to demonstrate the resource primitive and its usage in a real-world scenario.

9. **FR-9: MCP prompt.** The server must expose at least one MCP prompt to demonstrate the prompt primitive and its usage in a real-world scenario.

4.2.2 Non-Functional Requirements

1. **NFR-1: Safety.** The server must start in read-only mode by default. Write operations (create, update, delete) must only be registered when an explicit environment variable (`MCP_RHOAI_MODE=write`) is set.

2. **NFR-2: Host independence.** The server must communicate exclusively through the MCP standard and must not contain any host-specific code. It must be usable from any MCP-compatible host.

3. **NFR-3: Local cluster authentication.** The server does not manage its own authentication. The user must be logged in to an OpenShift cluster locally (via `oc login`) before starting the server. All Kubernetes API calls are executed using the credentials stored in the user's `kubeconfig` file, and therefore run with the permissions of the logged-in user.

4. **NFR-4: Code quality.** The codebase must pass static analysis and linting checks, and follow clear package separation.

5. **NFR-5: Unit testing.** A selected subset of tools must have unit tests.

6. **NFR-6: Evaluation.** End-to-end agent behaviour must be evaluated using an automated framework that measures tool selection

accuracy, parameter extraction correctness, and execution success rate.

4.3 Technology Choices

This section justifies and describes the key technology used for the implementation.

Go. The server is written in Go [4]. Go was chosen for two reasons. First, it is the dominant language in the Kubernetes ecosystem — Kubernetes itself, OpenShift, and the Kubeflow Notebook Controller are all written in Go, and the official Kubernetes client library (`client-go`) is a Go module. Using Go means the server can import `client-go` directly without a foreign-function interface or an HTTP wrapper. A Python or TypeScript implementation would need to communicate with the cluster through a generated REST client or a wrapper library rather than using the same native client that Kubernetes itself uses. Second, Go compiles to a single static binary with no runtime dependencies, which simplifies distribution — users only need to place the binary on their `PATH` and point their MCP host configuration at it. By contrast, a Python-based server would require the user to have a compatible Python interpreter and manage a virtual environment with the correct dependencies installed.

MCP Go SDK. The server uses the official MCP Go SDK [9] (version 1.1.0) for all protocol-level concerns: creating the MCP server instance, registering tools with their JSON Schema input definitions (derived automatically from Go struct tags), registering resources and prompts, and running the stdio transport loop. The SDK's typed tool registration function (`mcp.AddTool`) accepts a Go struct as the input type and automatically generates the JSON Schema that is advertised to the host, eliminating manual schema maintenance.

golangci-lint. Static analysis is performed by `golangci-lint` [3], a runner that aggregates multiple Go linters into a single invocation. It is integrated into the build process through the Makefile: `make build`

runs `golangci-lint run` before compiling the binary, ensuring that code quality checks pass on every build.

stdio transport. The server uses the stdio transport (Section 2.3.5) exclusively. This choice is driven by the deployment model: the MCP server runs as a local process on the same machine as the AI host, launched and managed by the host's MCP client configuration. The stdio transport requires no network configuration, no TLS certificates, and no separate service deployment, which keeps the setup minimal. If remote deployment is needed in the future, switching to the HTTP/SSE or Streamable HTTP transport would require changing only the transport initialisation in the entry point.

Promptfoo. End-to-end evaluation uses the Promptfoo framework [13], an open-source tool for testing LLM applications. Alternatives such as DeepEval were considered — DeepEval offers over fifty evaluation metrics, including a dedicated tool-correctness metric, but that breadth introduces complexity that is unnecessary when the only goal is to evaluate tool-calling behaviour. Promptfoo is more focused: its entire configuration fits into a single YAML file, it requires no custom evaluation code, and it directly supports the workflow needed here — sending natural-language prompts to an LLM, capturing the tool calls the model emits, and checking them against expected outcomes. It measures tool selection accuracy, parameter extraction correctness, and execution success rate. It also supports multiple LLM providers (OpenAI, Anthropic, Google), making it straightforward to compare tool-calling quality across models.

4.4 Architecture

This section describes the internal structure of the MCP server: how the codebase is organised, how the two operation modes are implemented, how the server communicates with the OpenShift cluster, and how the project is built.

4.4.1 Project Structure

The project root contains two key files: `main.go`, which is the entry point that creates the MCP server instance, registers tools, resources, and prompts, and starts the stdio transport loop; and a `Makefile` that drives the build pipeline described in Section 4.4.4. The remaining code is organised into four packages:

- `core` — shared type definitions (input/output structs for every tool), Kubernetes GroupVersionResource constants, and cluster authentication helpers.
- `tools` — the implementation of every MCP tool, plus a registry module that registers tools with the MCP server. The registry exposes two functions: `RegisterReadOnlyTools`, which registers the listing and monitoring tools that are always available, and `RegisterWriteTools`, which additionally registers the create, update, and delete tools. The entry point inspects the `MCP_RHOAI_MODE` environment variable and calls one or both functions accordingly.
- `resources` — the implementations of the two MCP resources: the Image Catalog and the Default Hardware Profile. A registry module registers both resources with the MCP server.
- `prompts` — the implementation of the Create Workbench MCP prompt, together with a registry module that registers it with the MCP server.

4.4.2 Two-Mode Design

The server's safety model is implemented at the registration level rather than at the execution level. When the server starts, the entry point checks the `MCP_RHOAI_MODE` environment variable. If the value is `write`, the server calls `RegisterWriteTools` followed by `RegisterReadOnlyTools`, making all tools available. For any other value (or when the variable is not set), only `RegisterReadOnlyTools` is called. Because tools that are not registered are never advertised to the host during capability negotiation, the LLM cannot invoke them — it does not know they exist. This approach is simpler and more robust than runtime permission checks, since there is no code path that could accidentally bypass a guard.

4.4.3 Kubernetes Client Strategy

The server uses two types of Kubernetes clients from the `client-go` library [22]. The **typed client** (`kubernetes.Clientset`) is used for operations on built-in resources such as pods and namespaces, where the Go type system provides compile-time safety. The **dynamic client** (`dynamic.DynamicClient`) is used for custom resources — Kubeflow Notebooks, OpenShift ImageStreams, and hardware profiles — where no generated Go types are available and the server must work with `unstructured.Unstructured` objects. Both clients are constructed from the user's `kubeconfig` file, reusing whatever credentials and permissions the user has when they run `oc login`.

To support unit testing, both clients are exposed as package-level function variables (`GetClientSet` and `GetDynamicClient`) that can be replaced with mock implementations in test code, satisfying NFR-5.

4.4.4 Build Pipeline

The project uses a Makefile to integrate all quality checks into a single build command. Running `make build` executes three stages in sequence: first `golangci-lint run` to verify that the code passes all configured linters, then `go test` to run the unit test suite, and finally `go build` to compile the binary. If any stage fails, the build stops immediately. This ensures that every produced binary has passed both linting and tests. Evaluation is run separately via `make eval`, which invokes Promptfoo against the compiled server.

5 Implementation

The previous chapter defined the requirements, technology choices, and architecture of the MCP server. This chapter describes how the server was implemented. It walks through the specific MCP tools, resources, and the prompt that were implemented. The chapter closes with a discussion of security considerations, code quality practices, and the evaluation methodology.

5.1 MCP Tools

The server exposes 24 MCP tools covering five resource domains. Each subsection below opens with a table listing its tools and then discusses the design decisions and non-trivial implementation details. In all tables, tools marked (*write*) are only registered when the environment variable `MCP_RHOAI_MODE=write` is set; all other tools are always available.

Each tool is registered with the MCP server using the SDK's `mcp.AddTool` function, which accepts a `mcp.Tool` descriptor (name and description) and a Go function that implements the tool's logic. Every tool function follows the same high-level pattern: obtain a Kubernetes client, perform the required API call, transform the result into a typed output struct, and return it. Errors are propagated back to the host as standard MCP error responses.

5.1.1 Workbench Management

Workbench management is the most complex part of the server, comprising six tools that operate on Kubeflow `Notebook` resources (API group `kubeflow.org/v1`) through the dynamic Kubernetes client.

Listing. The `list_workbenches` tool is the foundation on which several other tools are built. For each `Notebook` resource in the specified namespace, the tool reads RHOAI-specific annotations to extract the image display name, image tag, hardware profile name, and owner. Running status is determined by the presence of the `kubeflow-resource-stopped` annotation: if set, the workbench is stopped;

Table 5.1: Workbench management tools.

Tool	Description
`list_workbenches`	List workbenches in a namespace
`list_all_workbenches`	List workbenches across all namespaces
`create_workbench`	Create a new workbench (*write*)
`update_workbench`	Update image, hardware profile, or PVC (*write*)
`delete_workbench`	Delete a workbench (*write*)
`change_workbench_status`	Start or stop a workbench (*write*)

otherwise, it is running. For running workbenches, it additionally queries the typed client for pods labelled `notebook-name=<workbenchName>` to obtain uptime from the pod's start time. Disk usage is read from the PVC's requested storage capacity. The result is a list of `WorkbenchInfo` structs containing the workbench name, user, status, image, hardware profile, PVC name, namespace, uptime, and resource consumption (CPU, memory, disk, GPU). The `list_all_workbenches` tool reuses this logic with an empty namespace string, which in the Kubernetes API returns results across all namespaces, giving administrators a cluster-wide view.

Creation. The `create_workbench` tool is the most complex in the server. It must resolve the user-specified image display name and tag to a concrete container image URL by looking up the corresponding ImageStream, optionally create a PVC (defaulting to the workbench name and 10 GiB if none is provided), and fall back to the default hardware profile (Section 5.2) when no profile is specified. The manifest is constructed as an `unstructured.Unstructured` object with the same annotations the RHOAI dashboard would set — `opendatahub.io/image-display-name`, `notebooks.opendatahub.io/last-image-selecti` `opendatahub.io/hardware-profile-name`, among others. The container spec includes JupyterLab server arguments, two volume mounts (the user's PVC and a shared-memory `emptyDir`), and resource requests and limits derived from the hardware profile. After submission, the Notebook Controller creates the running environment. This design

ensures that workbenches created through the MCP server are indistinguishable from those created through the dashboard.

Updates and lifecycle control. The `update_workbench` tool supports three independent update dimensions — the notebook image, the hardware profile, and the attached PVC — each applied only when the corresponding input fields are non-empty, allowing partial updates. The `change_workbench_status` tool starts or stops a workbench by manipulating the `kubeflow-resource-stopped` annotation: setting it to the current timestamp stops the workbench; removing it (via a `nil` merge patch) restarts it. Before making any change, the tool checks the current state and returns early if the workbench is already in the requested state, avoiding redundant Kubernetes API calls. The `delete_workbench` tool issues a single `Delete` call; the Notebook Controller handles cleanup of the associated StatefulSet and pod, but the PVC is intentionally preserved so that the user's data is not lost.

5.1.2 Hardware Profiles

Table 5.2: Hardware profile tools.

Tool	Description
`list_hardware_profiles`	List all profiles in the RHOAI namespace
`create_hardware_profile`	Create a new profile (*write*)
`update_hardware_profile`	Merge-update profile resources (*write*)
`delete_hardware_profile`	Delete a profile (*write*)

Four tools manage hardware profiles (API group `infrastructure.opendatahub.io/v1`). The listing tool returns each profile's resource identifiers with their default, minimum, and maximum values. Creation and deletion are straightforward operations on `unstructured.Unstructured` objects.

The more interesting case is `update_hardware_profile`, which implements a *merge* strategy: it fetches the existing profile, iterates over the provided resource entries, and overwrites any entry whose resource identifier already exists while appending new ones. This means the agent can add a GPU to an existing profile without having to re-

specify the CPU and memory entries, which makes the tool more natural to use through conversational interaction.

5.1.3 Notebook Images

Table 5.3: Notebook image tools.

Tool	Description
`list_images`	List images with versions and dependencies
`create_custom_image`	Register a custom image (*write*)
`update_image`	Update display name or description (*write*)
`delete_image`	Delete with safety checks (*write*)

Notebook images are represented as OpenShift ImageStreams (API group `image.openshift.io/v1`) labelled with `opendatahub.io/notebook-image=true`. The `list_images` tool reuses the `GetImages` function from the `resources` package — the same function that backs the Image Catalog MCP resource (Section 5.2.1) — and formats each image's display name, container image URL, and version tags with their Python dependencies into a human-readable string. The `create_custom_image` tool registers a new ImageStream with a single `latest` tag, setting the labels and annotations required for dashboard visibility. The `update_image` tool modifies the display name or description annotations.

The most notable tool in this group is `delete_image`, which enforces two safety checks before proceeding. First, it verifies that the image is not a default RHOAI image by inspecting the `internal.config.kubernetes.io/previ` annotation — default images are managed by the platform and should not be removed by users. Second, it scans all workbenches across all namespaces and compares the `opendatahub.io/image-display-name` annotation to ensure the image is not currently in use. If either check fails, the deletion is rejected with an explanatory error message. These checks prevent accidental removal of images that would leave running workbenches in a broken state.

5.1.4 Storage and Namespaces

The `list_pvcs` tool lists all PVCs in a namespace, and the `list_namespaces` and `list_pods` tools provide basic cluster inspection (both registered

Table 5.4: Storage and infrastructure tools.

Tool	Description
`list_pvcs`	List PVCs in a namespace
`create_pvc`	Create a new PVC (*write*)
`update_pvc`	Resize a PVC (expansion only) (*write*)
`delete_pvc`	Delete a PVC (*write*)
`list_namespaces`	List cluster namespaces (*write*)
`list_pods`	List pods in a namespace (*write*)

only in write mode; `list_pods` is useful for debugging workbench issues such as `CrashLoopBackOff` states).

The `create_pvc` tool creates a PVC with the specified name and size. The `update_pvc` tool handles PVC resizing, comparing the requested size with the current size using the `resource.Quantity` type from the Kubernetes API toolkit. If the new size is smaller, the update is rejected with an error, since Kubernetes does not support shrinking PVCs. The `delete_pvc` tool removes a PVC by name.

5.1.5 Resource Consumption

Table 5.5: Resource consumption monitoring tools (all read-only).

Tool	Description
`list_resource_consumption_per_workbench`	Per-workbench consumption
`list_resource_consumption_per_namespace`	Aggregated per namespace
`list_resource_consumption_per_user`	Aggregated per user
`list_resource_consumption_per_cluster`	Aggregated cluster-wide

Four read-only tools provide resource consumption monitoring at different granularities: per workbench, per namespace, per user, and cluster-wide. All four report the same four resource types — CPU, memory, disk, and GPU — and are always available regardless of the server mode.

The values these tools report are based on Kubernetes resource *requests* declared in the workbench's container spec and the PVC's requested storage capacity, not on live metrics. This means they reflect what the workbench has *reserved*, which is the relevant quantity for capacity planning.

The per-workbench tool simply filters the output of the workbench listing logic by name. The three aggregating tools (per-namespace, per-user, and cluster-wide) all use a shared `sumResourceValues` helper that parses Kubernetes quantity strings (e.g. `4Gi`, `500m`), accumulates the numeric parts, and preserves the unit from the first non-zero value. The per-user tool filters by the `opendatahub.io/username` annotation across all namespaces, enabling administrators to answer questions such as "how many resources has user X reserved across the cluster?"

5.2 MCP Resources and Prompts

Beyond tools, the server exposes two MCP resources and one MCP prompt. Together with the tools described above, these demonstrate all three MCP server primitives in a real-world scenario.

5.2.1 Image Catalog Resource

The Image Catalog is registered with the URI `resource://mcp-server-rhoai/images` and the MIME type `application/json`. When read, it calls the `GetImages` function, which queries all ImageStreams in the default RHOAI namespace that carry the `opendatahub.io/notebook-image=true` label. For each image, it extracts the display name from annotations, the container image URL from the ImageStream's `status.dockerImageRepository` field, and all version tags with their Python dependency and software annotations. The result is serialised as a JSON array and returned to the host. An agent can read this resource before creating a workbench to discover which images are available and select a suitable one.

5.2.2 Default Hardware Profile Resource

The Default Hardware Profile is registered with the URI `resource://mcp-server-rhoai/ha` Unlike the Image Catalog, which queries the cluster dynamically, this resource returns a static default profile: 2 CPUs (min 1, max 4) and

4 GiB of memory (min 2 GiB, max 8 GiB). This profile is also used as the fallback by the `create_workbench` tool when no hardware profile is specified by the user. The resource gives the agent visibility into the default allocation, allowing it to inform the user about the resources a new workbench will receive if no explicit profile is provided.

5.2.3 Create Workbench Prompt

The Create Workbench prompt is registered under the name `create-workbench-prompt` with two required arguments: `namespace` and `name`. When invoked, it returns a single user-role message that instructs the agent to consult the Image Catalog resource to find a suitable Data Science image (preferring Python 3.12) and then call the `create_workbench` tool. This prompt demonstrates how the three MCP primitives interact in practice: a prompt guides the agent to first read a resource (the Image Catalog) and then invoke a tool (Create Workbench), orchestrating a multi-step workflow through a single user action.

5.3 Security

Security is addressed at two levels: the two-mode design described in Section 4.4.2, which prevents accidental cluster modifications, and the Kubernetes authentication model.

The server does not implement its own authentication or authorisation. Instead, it relies entirely on the user's existing OpenShift credentials. Both the typed and dynamic Kubernetes clients are constructed from the `kubeconfig` file at `~/.kube/config`, which stores the token obtained via `oc login`. Every API call the server makes therefore runs with the permissions of the logged-in user; if the user does not have permission to, for example, delete workbenches in a certain namespace, the Kubernetes API server rejects the request and the error is propagated back to the agent.

This delegation model has two advantages. First, it avoids introducing a separate credential management system — the server reuses the same authentication flow that platform engineers already use. Second, it ensures that the server cannot escalate privileges beyond what the

user already has, since every API call is subject to the same permission checks that the cluster enforces for all users.

The image deletion tool includes additional application-level safety checks. It refuses to delete images that are marked as RHOAI defaults (by inspecting the `internal.config.kubernetes.io/previousNamespaces` annotation) and images that are currently in use by any workbench (by scanning all workbenches across namespaces). These checks prevent accidental removal of images that would break existing workbenches.

5.4 Code Quality

Code quality is maintained through static analysis and unit testing, both integrated into the build pipeline.

5.4.1 Static Analysis

The project uses `golangci-lint` with five enabled linters: `errcheck` (ensures error return values are checked), `govet` (reports suspicious constructs), `ineffassign` (detects ineffectual assignments), `staticcheck` (a suite of advanced static analysis checks), and `unused` (finds unused code). Every build runs `golangci-lint run` as the first stage; if any linter reports an issue, the build fails immediately.

5.4.2 Unit Testing

Unit tests validate tool behaviour against a simulated OpenShift cluster without requiring access to a live environment. This is made possible by the injectable client design described in Section 4.4.3: the `GetClientSet` and `GetDynamicClient` function variables in the `tools` package are replaced with fake implementations in test code. The fakes use `fake.NewSimpleClientset` from `client-go` for the typed client and `dynamicfake.NewSimpleDynamicClient` for the dynamic client. Some tools that depend on other tools internally (for example, the resource consumption tools call the workbench listing logic) use additional function-level mocks — package-level function variables such as `listWorkbenchesFn` that are replaced in tests with simple functions returning predefined data.

Each test follows a three-step pattern: (1) save the original client factory and register a deferred restore, (2) create fake Kubernetes objects and inject a mock client, and (3) call the tool function under test and assert on the output. The test suite comprises eight test files covering every tool domain:

- **Workbenches** — tests for listing (stopped and running workbenches), creation (including automatic PVC creation), updates, deletion, and status changes. Pure helper functions such as image tag parsing and resource requirement construction are tested separately.
- **Hardware profiles** — CRUD operations and parsing of profile resource identifiers.
- **Notebook images** — creation, update, and deletion of custom images, including verification that default images and images in use by workbenches cannot be deleted. The `ListImages` tool and the underlying `GetImages` function (which backs the Image Catalog resource) are tested with fake ImageStream objects.
- **Storage** — PVC creation, deletion, resize (verifying that shrinking is rejected), and listing.
- **Namespaces and pods** — listing with typed client fakes.
- **Resource consumption** — the aggregation helpers (`parseResourceValue`, `sumResourceValues`) are tested directly, and all four granularity tools are tested with predefined workbench data.

5.5 Evaluation

End-to-end evaluation uses the Promptfoo framework [13] to measure how well an LLM agent uses the MCP server's tools in response to natural-language requests. Promptfoo was chosen for its simplicity — the entire evaluation is configured in a single YAML file (`promptfoo.yaml`) with no custom evaluation code required, and it can be run with `make eval`.

5.5.1 Evaluation Setup

Promptfoo connects to the compiled MCP server binary as an MCP provider, starting the server with `MCP_RHOAI_MODE=write` so that all

tools are available. The active provider in the configuration is Google's Gemini 2.5 Pro; configurations for OpenAI GPT-4o, GPT-4o-mini, and Anthropic Claude 3.5 Sonnet are included as comments, making it straightforward to compare tool-calling quality across models.

Each test case consists of a natural-language prompt (the `prompt` variable) and one or more assertions. The assertions use Promptfoo's `tool-call-f1` metric, which checks whether the set of tools the model actually called matches the expected set. This metric captures both *precision* (the model did not call unnecessary tools) and *recall* (the model called all required tools), combining them into a single F1 score.

5.5.2 Test Cases

The evaluation suite includes four test cases that exercise different parts of the server:

1. **List all workbenches.** Prompt: "Show me all workbenches across all namespaces." Expected tool: `list_all_workbenches`. Tests whether the model picks the cross-namespace variant over the single-namespace one.

2. **Resource consumption per namespace.** Prompt: "What is the resource consumption in test-namespace?" Expected tool: `list_resource_consumption`. Tests whether the model correctly identifies the namespace-level consumption tool from the four available granularities.

3. **List workbenches in a namespace.** Prompt: "List workbenches in test-namespace." Expected tool: `list_workbenches`. Tests the single-namespace listing.

4. **Create a workbench.** Prompt: "Create a workbench named mynotebook in namespace test-namespace using Jupyter image." Expected tool: `create_workbench`. Tests whether the model selects the creation tool and extracts the correct parameters.

A 2-second delay is configured between test cases to avoid API rate limiting.

5.5.3 Running the Evaluation

The evaluation is invoked via `make eval`, which runs `npx promptfoo@latest eval -c promptfoo.yaml`. Results are written to `promptfoo-results.json` and can also be explored in a web-based dashboard via `make eval-view`. The YAML-based configuration makes it straightforward to add new test cases, switch providers, or adjust assertion types as the tool set evolves.

6 Conclusion

6.1 Future Work: Notebooks 2.0

Bibliography

[1] Anthropic. *Introducing the Model Context Protocol*. Nov. 25, 2024. URL: https://www.anthropic.com/news/model-context-protocol (visited on 03/10/2026).

[2] Anthropic. *Model Context Protocol Specification*. 2024. URL: https://modelcontextprotocol.io/specification/latest (visited on 03/10/2026).

[3] golangci-lint Contributors. *golangci-lint — Fast Go linters runner*. 2024. URL: https://golangci-lint.run (visited on 03/22/2026).

[4] Google. *The Go Programming Language*. 2009. URL: https://go.dev (visited on 03/22/2026).

[5] JSON-RPC Working Group. *JSON-RPC 2.0 Specification*. 2013. URL: https://www.jsonrpc.org/specification (visited on 03/10/2026).

[6] Kubeflow Contributors. *Kubeflow Documentation*. 2024. URL: https://www.kubeflow.org/docs/ (visited on 03/22/2026).

[7] Kubeflow Contributors. *Notebook CRD — Kubeflow GitHub Repository*. 2024. URL: https://github.com/kubeflow/kubeflow/blob/master/components/notebook-controller/config/crd/bases/kubeflow.org_notebooks.yaml (visited on 03/22/2026).

[8] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474. URL: https://arxiv.org/abs/2005.11401.

[9] Model Context Protocol Contributors. *MCP Go SDK — GitHub Repository*. 2025. URL: https://github.com/modelcontextprotocol/go-sdk (visited on 03/22/2026).

[10] Model Context Protocol Contributors. *Model Context Protocol — GitHub Organisation*. 2024. URL: https://github.com/modelcontextprotocol (visited on 03/10/2026).

[11] Open Data Hub Contributors. *Open Data Hub Documentation*. 2024. URL: https://opendatahub.io/docs/ (visited on 03/22/2026).

[12] Long Ouyang et al. "Training language models to follow instructions with human feedback". In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 27730–27744. URL: https://arxiv.org/abs/2203.02155.

[13] Promptfoo Contributors. *Promptfoo: Test and evaluate LLMs*. 2024. URL: https://www.promptfoo.dev (visited on 03/10/2026).

[14] Red Hat. *Configuring Routes — OpenShift Container Platform Documentation*. 2024. URL: https://docs.openshift.com/container-platform/latest/networking/routes/route-configuration.html (visited on 03/22/2026).

[15] Red Hat. *Managing Image Streams — OpenShift Container Platform Documentation*. 2024. URL: https://docs.openshift.com/container-platform/latest/openshift_images/image-streams-manage.html (visited on 03/22/2026).

[16] Red Hat. *OpenShift Container Platform Documentation*. 2024. URL: https://docs.openshift.com/container-platform/latest/welcome/index.html (visited on 03/22/2026).

[17] Red Hat. *Red Hat OpenShift AI Documentation*. 2024. URL: https://docs.redhat.com/en/documentation/red_hat_openshift_ai_self-managed/ (visited on 03/22/2026).

[18] Red Hat. *Understanding Authentication — OpenShift Container Platform Documentation*. 2024. URL: https://docs.openshift.com/container-platform/latest/authentication/understanding-authentication.html (visited on 03/22/2026).

[19] Red Hat. *Understanding Operators — OpenShift Container Platform Documentation*. 2024. URL: https://docs.openshift.com/container-platform/latest/operators/understanding/olm-what-operators-are.html (visited on 03/22/2026).

[20] Red Hat. *Working with Projects — OpenShift Container Platform Documentation*. 2024. URL: https://docs.openshift.com/container-platform/latest/applications/projects/working-with-projects.html (visited on 03/22/2026).

[21] Red Hat. *Working with Workbenches — Red Hat OpenShift AI Documentation*. 2024. URL: https://docs.redhat.com/en/documentation/red_hat_openshift_ai_self-managed/2-latest/html/working_on_data_science_projects/using-project-workbenches_workbenches (visited on 03/22/2026).

[22] The Kubernetes Authors. *client-go — Go client for Kubernetes*. 2024. URL: https://github.com/kubernetes/client-go (visited on 03/22/2026).

[23] The Kubernetes Authors. *Custom Resources — Kubernetes Documentation*. 2024. URL: https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/ (visited on 03/22/2026).

[24] The Kubernetes Authors. *Kubernetes Documentation*. 2024. URL: https://kubernetes.io/docs/home/ (visited on 03/22/2026).

[25] The Kubernetes Authors. *Labels and Selectors — Kubernetes Documentation*. 2024. URL: https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/ (visited on 03/22/2026).

[26] The Kubernetes Authors. *Namespaces — Kubernetes Documentation*. 2024. URL: https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/ (visited on 03/22/2026).

[27] The Kubernetes Authors. *Persistent Volumes — Kubernetes Documentation*. 2024. URL: https://kubernetes.io/docs/concepts/storage/persistent-volumes/ (visited on 03/22/2026).

[28] The Kubernetes Authors. *Pods — Kubernetes Documentation*. 2024. URL: https://kubernetes.io/docs/concepts/workloads/pods/ (visited on 03/22/2026).

[29] The Kubernetes Authors. *StatefulSets — Kubernetes Documentation*. 2024. URL: https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/ (visited on 03/22/2026).

[30] Ashish Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems* 30 (2017). URL: https://arxiv.org/abs/1706.03762.

[31] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations*. 2023. URL: https://arxiv.org/abs/2210.03629.